CASE STUDY

PHP 8.4.27, CVE-2026-109618 and CVE-2026-109619: Two Fixes Verified in the Lab

A TeamStarWolf case study. Published 4 November 2026; every source checked as of that date.

Executive summary

PHP is the runtime under a large part of the community's web: forums, wikis, content-management systems, small-business shops and the admin panels inside appliances. On 21 October 2026 the PHP project shipped security releases (8.2.35, 8.3.36, 8.4.27 and 8.5.12) for two flaws in the interpreter itself, CVE-2026-109618 and CVE-2026-109619, which CISA's alert calls "php2shell". Both let an unauthenticated attacker run code on the server (as the web worker account, in the vendor's 109618 wording), and the vendor validated that on Apache mod_php, where one of them fires before any application code runs. CISA added both to the Known Exploited Vulnerabilities catalog on 26 October with a three-day due date.

This study is the lab companion to PHP2Shell and the October 2026 Attack Wave, which is the home for the October wave narrative, the exploitation telemetry and the generic per-platform remediation table. Here the subject is narrower: what each flaw is in the vendor's own words, the dated public record of the fix, and what a two-container lab run on 4 November 2026 showed when a vulnerable `php:8.4.26-apache` image ran beside a fixed `php:8.4.27-apache` image. The lab was version-check class. No proof-of-concept was run; it recorded how the two builds differ to an observer, whether the vendor's interim workarounds take effect, where common inventory methods miss a vulnerable PHP, and that cleanup left nothing behind.

> Evidence grade: the vulnerabilities, the fix and the KEV listing are **Confirmed** from the vendor, CVE, NVD and CISA records; the lab readings are **Demonstrated** from the captures named in each section; in-the-wild telemetry from third parties is **Reported**.

Key facts at a glance

Field	Value
Identifiers	CVE-2026-109618, "RCE via OOM corruption" (GHSA-w8hw-w9fx-j79w); CVE-2026-109619, "RCE via stale global state" (GHSA-6wmf-4rv5-p3rp) [1] [2] (**Confirmed**)
Shared name	"php2shell", used by the CISA alert [17] (**Confirmed**)
Class	Memory corruption after an out-of-memory abort (109618); use after free of worker-global state across requests (109619) [1] [2] (**Confirmed**)
Weaknesses	CWE-416, CWE-755, CWE-824, CWE-1325 and CWE-1419 in the advisories and KEV; NVD lists the first four [1] [2] [5] [18] (**Confirmed**)
CVSS	CNA v4.0 9.2 Critical for both; NVD v3.1 9.8 Critical (109618) and 8.1 High (109619) [5] [6] (**Confirmed**)
Access required	None: network, unauthenticated, no user interaction [1] [2] [5] (**Confirmed**)
Affected (CNA)	8.2 before 8.2.35, 8.3 before 8.3.36, 8.4 before 8.4.27, 8.5 before 8.5.12; NVD's CPE ranges start at 7.3.0 (109618) and 5.4.0 (109619) [3] [5] [6] (**Confirmed**)
Fixed	8.2.35, 8.3.36, 8.4.27, 8.5.12 (announced 21 October 2026); 8.6.0 RC4 for testers; source backports for end-of-life 7.4, 8.0 and 8.1 [8] [9] [10] (**Confirmed**)
Highest-risk setup	Apache mod_php; FPM, php-cgi, FrankenPHP and LiteSpeed are also at risk [9] (**Confirmed**)
In the wild	KEV since 26 October 2026, due 29 October; CISA ADP SSVC "exploitation active" [6] [18] (**Confirmed**); attempts from 17 October by Amazon's count [22] (**Reported**)
Lab result	Vulnerable and fixed builds told apart by `php -v`, the `X-Powered-By` header, the Apache start-up line and a marker page; both interim workarounds took effect; cleanup verified [46] [47] (**Demonstrated**)

Key findings

- **CVE-2026-109618 is reachable before the application runs.** PHP aborts on out-of-memory with `longjmp()`, and some request memory is reachable before it is initialized, so request shutdown then reads or branches on invalid state. On mod_php, request shutdown always runs even after a failed start-up, so the attack happens before any application code. The vendor validated unauthenticated remote code execution over HTTP against mod_php with default limits, and calls FPM susceptible by source analysis only (**Confirmed** [1]).
- **CVE-2026-109619 needs a producer and a consumer on the same worker.** Worker-global state keeps pointing at freed request memory after a fatal error or a late diagnostic, and a later request on the same worker uses it before initialization. Session upload progress is one confirmed consumer, it is on by default, and it needs no `session_start()`; RCE was validated on mod_php and FPM (**Confirmed** [2]).
- **The fix changed how PHP fails, not one line.** Commit `3584a98bda0f` ("Avoid state corruption via bailout") replaces the bailout on OOM with `UnwindExit` and resets globals that start-up can read [14]; the advisory describes the design as a delayed soft OOM at the next VM safe point plus a hard `_exit()` at a second, higher threshold [1]. It adds 10,759 lines and removes 988 (**Confirmed** [1] [14]).

- **Exploitation attempts came before the patch.** Amazon reports an attempt against its MadPot sensors on 17 October, four days before disclosure (**Reported** [22]). The PHP Foundation reports that the CVE ids leaked on 20 October with a wrong 10.0 score, which pulled the release forward to 21 October (**Confirmed** [21]).
- **In the lab, the upgrade was the only thing a scanner could see.** The mitigated 8.4.26 container still sent `X-Powered-By: PHP/8.4.26`, and the fixed 8.4.27 container still had session upload progress on by default (**Demonstrated** [46] [47]). A version-based scanner is right to keep flagging a mitigated host, and an upgraded host still needs the upload-progress setting turned off until the next bugfix release, as php.net advises (**Confirmed** [11]).
- **Common inventory methods missed the vulnerable build.** Inside the official Docker image, `dpkg -l` returned no PHP rows and `rpm` was absent; only `php -v`, the `libphp.so` module file and the header showed the build. A header check against `/` returned no `X-Powered-By` when no PHP script was served there (**Demonstrated** [47]).
- **Backported distribution builds look old.** Ubuntu noble's fixed package is 8.3.6-based and RHEL 9's default `php` package is 8.0.30-based, so a banner check flags patched hosts (**Confirmed** [32] [33]; consequence **Inferred**).

Top recommendations

1. Update every PHP that serves web requests to 8.2.35, 8.3.36, 8.4.27 or 8.5.12 or later, or to your distribution's fixed package, and restart every worker. The vendor calls updating the only safe choice [8].
2. Inventory by binary and by served URL, not by package manager alone: `php -v` inside each container, the loaded Apache module, and a header read against a URL that PHP actually handles (Part 4).
3. Keep `session.upload_progress.enabled=0` in `php.ini`, the FPM pool or the server configuration after updating, until the next regular bugfix release, as php.net and NCSC-NL advise [11] [20].
4. If you cannot update today, apply the vendor's interim settings for both flaws, administratively, and restart every worker. They are not a complete fix [1] [2] [9].
5. Treat any PHP host that was internet-facing from 17 October as a candidate for investigation: KEV marks both entries for forensic triage, and Amazon advises looking back to that date [18] [22].
6. Rebuild container images from a fixed base image pinned by digest, because the fixed `php:8.4.27-apache` tag was last pushed on 31 October, after the PHP release [30].

Scope and method

This study covers CVE-2026-109618 and CVE-2026-109619 as fixed in PHP 8.4.27 and the sibling releases of 21 October 2026, the public record from the 2004 and 2020 root causes the finder cites to 4 November 2026, and one lab run on 4 November 2026. Every web source was opened on 4 November 2026; times are UTC unless a source states otherwise. Quotations are short and attributed. Individual developers and researchers are credited through their organizations.

Each claim carries one of four grades.

Grade	Meaning
Confirmed	Taken from a primary source that was opened: the PHP project, the CVE or NVD record, a government agency, a distribution's own tracker, or the reporting researcher's own advisory text as published by the vendor
Reported	A single third party's account or telemetry, such as a honeypot operator or a researcher's own write-up, not independently corroborated here
Demonstrated	Shown in this study's own lab captures, with the capture file named, on images anyone can pull and rerun
Inferred	The study's own reasoning from cited facts, labelled as such

The lab. The run used Docker on a lab host, with two containers per CVE plan: `php:8.4.26-apache` (vulnerable) and `php:8.4.27-apache` (fixed), each published only on a loopback port (127.0.0.1:8081 and 127.0.0.1:8082). Both plans were version-check class: they read versions, settings and headers, applied the vendor's interim workarounds, and removed everything afterwards. The logs record only version-check traffic, HEAD and GET requests from `curl/8.22.0` returning 200, and no OOM, crash, segfault or HTTP 500 (**Demonstrated** [47], `02-test.txt`). The screenshots are renderings of the captured command output with a neutral `lab$` prompt and the lab host anonymized. Captures for the two plans are sources [46] (CVE-2026-109618) and [47] (CVE-2026-109619).

Part 1: the two flaws

CVE-2026-109618: RCE via OOM corruption

PHP has a per-request memory limit. When a request crosses it, PHP has historically aborted with `longjmp()`, jumping straight out of whatever it was doing. The vendor advisory explains the problem with that design: in some contexts request memory is reachable before it has been initialized, so an abort at the wrong moment leaves invalid state behind, and request shutdown then reads or branches on it (**Confirmed** [1]).

The advisory names three ways a request can drive PHP into that out-of-memory abort during multipart parsing (**Confirmed** [1]):

- a request without a usable declared content length passing the `post_max_size` check;
- unlimited headers per multipart part;
- empty file controls that skip `max_file_uploads`, while nested field names multiply the array work.

Where it is reachable depends on the server API. Apache mod_php always runs request shutdown, even after a failed start-up, so the attack happens before any application code runs. The vendor validated unauthenticated remote code execution as the web worker account over HTTP against mod_php with default input and memory limits. FPM is susceptible by source analysis, but no complete FPM exploit has been shown (**Confirmed** [1]). The finder's write-up, from Calif, covers the pair: it dates the earliest affected path to PHP 5.4.0 (NVD starts CVE-2026-109618 itself at 7.3.0), describes two OOM producers and three interruptible points, and traces the issue to a 2004 memory-limit bug and PHP bug #78876 from 2020 (**Reported** [26]).

CVE-2026-109619: RCE via stale global state

The second flaw is a use after free that spans requests. After a fatal error or a late diagnostic, worker-global error, session, object or extension state can keep pointing at request memory that has been freed. A later request on the same worker then uses that state before it is initialized. The flaw needs a producer, which leaves the stale pointer behind, and a consumer, which reads it, on the same worker (**Confirmed** [2]).

The vendor confirms two early consumers (**Confirmed** [2]):

- **INI diagnostics applied before** `php_request_startup()`, through mod_php `.htaccess` files and Apache `php_value`, `php_flag` and `php_admin_*` directory configuration;
- **session upload progress**, which is on by default and needs no `session_start()`.

The confirmed cases are PHP 8.5's `fatal_error_backtraces` (RCE on mod_php) and the session upload progress lifecycle (RCE validated on mod_php and FPM). Suppressing errors, by changing `error_reporting` or `display_errors` or by using `@`, does not protect (**Confirmed** [2]). Wiz Research, one of the reporters, demonstrated pre-authentication RCE on WordPress, Drupal and Joomla container images and remote disclosure of worker memory through `PHPSESSID` cookies, and says every version back to 5.4 is hit by at least one issue (**Reported** [25]).

The fix

Both flaws are closed by the same change. Commit `3584a98bda0f` ("Avoid state corruption via bailout") was authored on 18 October 2026 and committed on 21 October at 14:06 UTC. Its message describes replacing the bailout on OOM with `UnwindExit` and resetting globals that can be read during start-up; it names both advisories and both CVEs, and it adds 10,759 lines while removing 988 (**Confirmed** [14]). The advisory gives the design: `longjmp()` on OOM is replaced by a delayed soft OOM at the next VM safe point, plus an immediate hard OOM through `_exit()` at a second, higher threshold (**Confirmed** [1]).

The PHP 8 ChangeLog dates Version 8.4.27 to 22 October 2026, and its first Core entry fixes GHSA-w8hw-w9fx-j79w and GHSA-6wmf-4rv5-p3rp. The previous release, 8.4.26, is dated 24 September 2026 (**Confirmed** [13]). The release announcement calls 8.4.27 a security release and says there are no complete mitigations, so updating is the only safe choice (**Confirmed** [8]). PHP 8.6.0 RC4, published the same day, carries the fixes for pre-release testers (**Confirmed** [10]). For the end-of-life 7.4, 8.0 and 8.1 branches, the security team published minimal source backports, and Linux distributions and long-term-support vendors received the patches in advance (**Confirmed** [9]).

Scores, weaknesses and affected ranges

The records agree on the fixed versions, the CNA score and the SSVC decision, and differ on the NVD v3.1 score, the CWE list and the lower bound of the affected range (**Confirmed** [1] [2] [3] [4] [5] [6] [18]):

Field	CVE-2026-109618	CVE-2026-109619
CNA CVSS v4.0	9.2 Critical, `AV:N/AC:H/AT:P/PR:N/UI:N/VC:H/VI:H/VA:H`	9.2 Critical
NVD CVSS v3.1	9.8 Critical, `AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H`	8.1 High, `AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:H`

Field	CVE-2026-109618	CVE-2026-109619
CISA ADP SSVC (21 October)	Exploitation active, automatable no, technical impact total	Same
CWE (advisory and KEV)	CWE-416, 755, 824, 1325, 1419	Same
CWE (NVD)	CWE-416, 755, 824, 1325	Same
Affected (CNA, default unaffected)	8.2 before 8.2.35; 8.3 before 8.3.36; 8.4 before 8.4.27; 8.5 before 8.5.12	Same
NVD CPE lower bound	7.3.0	5.4.0
NVD status	Analyzed, last modified 2026-11-02	Analyzed, last modified 2026-11-02
KEV name	PHP Memory Corruption Vulnerability	PHP Stale Internal Request-State Vulnerability

The CNA range lists only the supported branches, so its lower bound says nothing about older PHP. NVD's CPE ranges go back to 7.3.0 for CVE-2026-109618 and 5.4.0 for CVE-2026-109619 (**Confirmed** [5] [6]); the 5.4.0 bound agrees with the finder's and Wiz's statements about 5.4, while the 7.3.0 bound for 109618 starts later than the earliest path Calif describes (**Reported** [25] [26]). For an inventory, treat every PHP older than the fixed release on its branch, including end-of-life branches, as affected unless a distribution says otherwise (**Inferred**). NVD's CPE list also names downstream products, Siemens (RUGGEDCOM APE1808, Spectrum Power and others) and Palo Alto PAN-OS and Panorama (**Confirmed** [5]); those vendors' own advisories were not opened for this study.

Part 2: the public record

The October wave study covers the exploitation wave and the wider month. This timeline is the dated record of these two CVEs and the 8.4.27 fix.

Date (UTC)	Event	Grade	Source
2004 and 2020	A memory-limit bug (2004) and PHP bug #78876 (2020), which the finder traces as the roots of the OOM issue	Reported	[26]
2024	The OOM problem is first noticed while a Quarkslab audit finding is being fixed	Confirmed	[21]
2026-07-30	Calif begins an audit	Confirmed	[21]
2026-08-11	Calif reports unconditional RCE	Confirmed	[21]
2026-08-19	Wiz finds its first bug	Reported	[25]
2026-08-24	Wiz reports to PHP	Confirmed	[21] [25]
2026-09-18	OpenAI sends more vectors	Confirmed	[21]
2026-09-21	PHP acknowledges Wiz's report	Reported	[25]

Date (UTC)	Event	Grade	Source
2026-09-24	PHP 8.4.26 released (the lab's vulnerable build)	Confirmed	[13]
2026-10-12	The PHP CNA reserves both CVE ids	Confirmed	[3] [4]
2026-10-17	Amazon sees a php2shell attempt against MadPot from an anonymization network	Reported	[22]
2026-10-18	Fix commit `3584a98bda0f` authored	Confirmed	[14]
2026-10-20	php.net gives advance notice of a critical security release for 21 October at 14:00 UTC, two vulnerabilities at CVSS v4.0 9.2; CERT-EU advisory 2026-018 v1.0	Confirmed	[7] [19]
2026-10-20, about 15:00	The CVE ids leak with a wrong 10.0 score; the release moves from 22 October to 21 October 14:00 UTC	Confirmed	[21]
2026-10-21, about 02:00	More attempts against MadPot	Reported	[22]
2026-10-21, 07:07	`php:8.4.26-apache` last pushed to Docker Hub	Confirmed	[29]
2026-10-21, 07:41	A distributor publishes fixed images ahead of the coordinated release	Confirmed	[21]
2026-10-21, 08:37	First of the exploitation requests TU Delft counts	Reported	[23]
2026-10-21, 13:30	Release time stated by CERT-EU	Confirmed	[19]
2026-10-21, 13:34	Both GitHub Security Advisories published	Confirmed	[1] [2]
2026-10-21, 13:48 and 13:54	CVE-2026-109618 and CVE-2026-109619 published, crediting Calif.io, Wiz and OpenAI as finders	Confirmed	[3] [4]
2026-10-21, 14:06	Fix commit committed	Confirmed	[14]
2026-10-21	8.4.27 release announcement and php.net security advisory; 8.6.0 RC4; oss-security posts from the PHP security team; CISA alert; CISA ADP SSVC "exploitation active"; CERT-EU v1.1; watchTowr reports both exploited against its honeypots	Confirmed (watchTowr Reported)	[8] [9] [10] [15] [16] [17] [6] [19] [24]
2026-10-22	ChangeLog date for Version 8.4.27	Confirmed	[13]
2026-10-23	CISA revises its alert and adds a SIGMA rule; php.net informational advisory for follow-on CVE-2026-115560; CERT-EU v1.2	Confirmed	[17] [35] [11] [19]
2026-10-26	Both CVEs added to KEV, due 29 October; CISA revises its alert again; php.net publishes Windows rebuilds	Confirmed	[18] [17] [12]
2026-10-28	The amd64 image behind `php:8.4.27-apache` pushed; Corelight's detection package dated	Confirmed	[30] [36]

Date (UTC)	Event	Grade	Source
2026-10-31, 12:57	`php:8.4.27-apache` tag last pushed	Confirmed	[30]
2026-11-02	NVD last modifies both records	Confirmed	[5] [6]
2026-11-03	GHSA-w8hw-w9fx-j79w last updated; KEV catalog version 2026.11.03	Confirmed	[1] [18]
2026-11-04, 04:51 to 04:55	Lab run (container log timestamps)	Demonstrated	[46] [47]
2026-11-04	TU Delft count closes at 06:13; NCSC-NL tracks 930 products that embed or depend on PHP, 215 vulnerable and 167 fixed	Reported (TU Delft); Confirmed (NCSC-NL)	[23] [20]

The two oss-security posts state the impact in one line each: the 109618 post says unauthenticated RCE was demonstrated against mod_php before application code runs, and the 109619 post says RCE was demonstrated against mod_php and PHP-FPM (**Confirmed** [15] [16]). Neither CVE has a public php-src issue of its own; the tracker entries are the two GitHub Security Advisories (**Inferred** from the advisory pages and the issue tracker [1] [2]).

Where the sources disagree

- **The release time.** php.net planned 14:00 UTC [7]; CERT-EU states 13:30 UTC [19]; the advisories went up at 13:34 UTC [1] [2]; the fix commit landed at 14:06 UTC [14]. Of the stated release times, CERT-EU's 13:30 UTC is the earliest, although a distributor's fixed images were public from 07:41 UTC [21] (**Inferred**).
- **The release date.** The ChangeLog dates 8.4.27 to 22 October [13], one day after the announcement [8]. The PHP Foundation reports that the release had been planned for 22 October and moved forward after the leak [21], which likely explains the ChangeLog date (**Inferred**).
- **The weaknesses.** CWE-1419 is in the advisories and KEV but not in NVD's list [1] [5] [18].
- **The severity.** NVD's v3.1 vector for 109619 has `AC:H`, giving 8.1, against 9.8 for 109618; both CNA v4.0 scores are 9.2 [5] [6].
- **The affected range.** The CNA lists supported branches from 8.2; NVD starts at 7.3.0 and 5.4.0 [3] [5] [6].
- **How the wave study frames 109619.** The October wave study describes CVE-2026-109619 as a use after free in the upload-progress feature [44]. The vendor's own description is broader: stale worker-global state with two confirmed early consumers, of which upload progress is one [2]. This study follows the vendor's grouping.

After the release

Three follow-ons matter for anyone checking hosts in November (**Confirmed** unless marked):

- **CVE-2026-115560.** On 23 October php.net issued an informational advisory for a follow-on in session upload progress, which needs a runtime switch of `session.serialize_handler` to `php`, and advised keeping `session.upload_progress.enabled=0` until the next regular bugfix release [11].
- **Windows rebuilds.** On 26 October php.net published Windows builds 8.5.12-1, 8.4.27-1, 8.3.36-1 and 8.2.35-1 with Brotli 1.2.1 for CVE-2026-111497, the Brotli decoder flaw covered in the Node.js

CVE-2026-111497 lab study. The PHP runtime version is unchanged; the `-1` suffix marks only the package rebuild [12].

- **Regressions reported by the community.** Two open, community-filed php-src issues report regressions in 8.5.12 and 8.4.27: some non-fallible allocations now end the process with `_exit(137)` at the new hard memory limit instead of raising a catchable fatal error (GH-24492), and the upload-progress session decoder that now runs before request start-up destroys sessions holding objects with reserved `__PHP_Incomplete_*` property names (GH-24493) (**Reported** [27] [28]). Neither is a reason to stay on 8.4.26; both are reasons to watch worker exits and session errors after the upgrade (**Inferred**).

Part 3: what the lab showed

Setup

The lab pulled the two official images and recorded their digests: `php:8.4.26-apache` at `sha256:60111400ff77...c9c11a` and `php:8.4.27-apache` at `sha256:4af38a024599...641052` (**Demonstrated** [46] [47], `01-setup.txt`). Both digests match the live Docker Hub tags on 4 November: the 8.4.26 tag was last pushed on 21 October; the fixed `php:8.4.27-apache` tag was last pushed on 31 October, ten days after the PHP release, and its amd64 image is dated 28 October (**Confirmed** [29] [30]). Each container got a one-line PHP page: the CVE-2026-109618 plan wrote `index.php` printing `PHP_VERSION`, and the CVE-2026-109619 plan wrote a `version.php` marker page printing `LAB-MARKER php=` and the version.

CVE-2026-109619 setup

```
lab$  docker ps --format '{{.Names}}'

# Pull the confirmed vulnerable mod_php image (Docker Hub tag confirmed 2026-11-03)
lab$  docker pull php:8.4.26-apache
Error response from daemon: Head "https://registry-1.docker.io/v2/library/php/manifests/8.4.26-apache": read tcp 198

# Pull the confirmed fixed mod_php image for side-by-side comparison
lab$  docker pull php:8.4.27-apache
8.4.27-apache: Pulling from library/php
Digest: sha256:4af38a0245996e700e20ec211b9f1c44fb789ad8cc7d24c7ea81ff382b641052
Status: Image is up to date for php:8.4.27-apache
docker.io/library/php:8.4.27-apache

# Create an isolated Docker network with no outbound route for the two targets
lab$  docker network create --internal php2shell-lab
e66ce01a87dc0186e72f775ea1065bcfbafdbef52a66357a6b29ccc9839a022a

# Start the vulnerable target, published only on the attacker VM loopback port 8081
lab$  docker run -d --name php2shell-vuln --network php2shell-lab -p 127.0.0.1:8081:80 php:8.4.26-apache
d4d211b33ba8929ebb8c96aca94e6ac788609296ed719a6a8cac97d344427515

# Connect the vulnerable target to the default bridge too, so the 127.0.0.1 port publish works (the internal network
lab$  docker network connect bridge php2shell-vuln

# Start the fixed target, published only on loopback port 8082
lab$  docker run -d --name php2shell-fixed --network php2shell-lab -p 127.0.0.1:8082:80 php:8.4.27-apache
e0f4779a2955d17b32058b193cce8072b414a2c49cf7b4cefab3102e2cf2c0a4

# Connect the fixed target to the default bridge for its loopback publish
lab$  docker network connect bridge php2shell-fixed

# Put a harmless one-line version page inside each container (marker content only, no input handling)
lab$  for c in php2shell-vuln php2shell-fixed; do docker exec $c sh -c 'echo "<?php echo \"LAB-MARKER php=\" . PHP_V

# The re-pull of php:8.4.26-apache above hit a transient registry reset; the pinned tag was already pulled in attemp
lab$  docker pull php:8.4.26-apache
8.4.26-apache: Pulling from library/php
Digest: sha256:60111400ff77bdc44fa79b4f072afe49a42abba71e0846ad496be4761dc9c11a
Status: Image is up to date for php:8.4.26-apache
docker.io/library/php:8.4.26-apache

lab$  docker image inspect --format '{{.RepoTags}} {{index .RepoDigests 0}}' php:8.4.26-apache php:8.4.27-apache
[php:8.4.26-apache] php@sha256:60111400ff77bdc44fa79b4f072afe49a42abba71e0846ad496be4761dc9c11a
[php:8.4.27-apache] php@sha256:4af38a0245996e700e20ec211b9f1c44fb789ad8cc7d24c7ea81ff382b641052

lab$  docker ps --format '{{.Names}} {{.Image}} {{.Ports}}'
php2shell-fixed php:8.4.27-apache 127.0.0.1:8082->80/tcp
php2shell-vuln php:8.4.26-apache 127.0.0.1:8081->80/tcp
```

Capture `01-setup.txt` of the CVE-2026-109619 run [47], 4 November 2026 UTC: both images pinned by digest, the lab network and both containers published on loopback only. Long lines are cut at the image's right edge.

Both containers are Apache mod_php targets, the highest-risk setup in the php.net advisory [9]: `apache2ctl -M` listed `php_module (shared)` in each, and the module file is `/usr/lib/apache2/modules/libphp.so` (**Demonstrated** [46] `03-exposure.txt`; [47] `02-test.txt`, `03-exposure.txt`).

The observable that differs

One upgrade fixes both CVEs, so both plans looked for the same difference (**Demonstrated** [46] [47], `02-test.txt`):

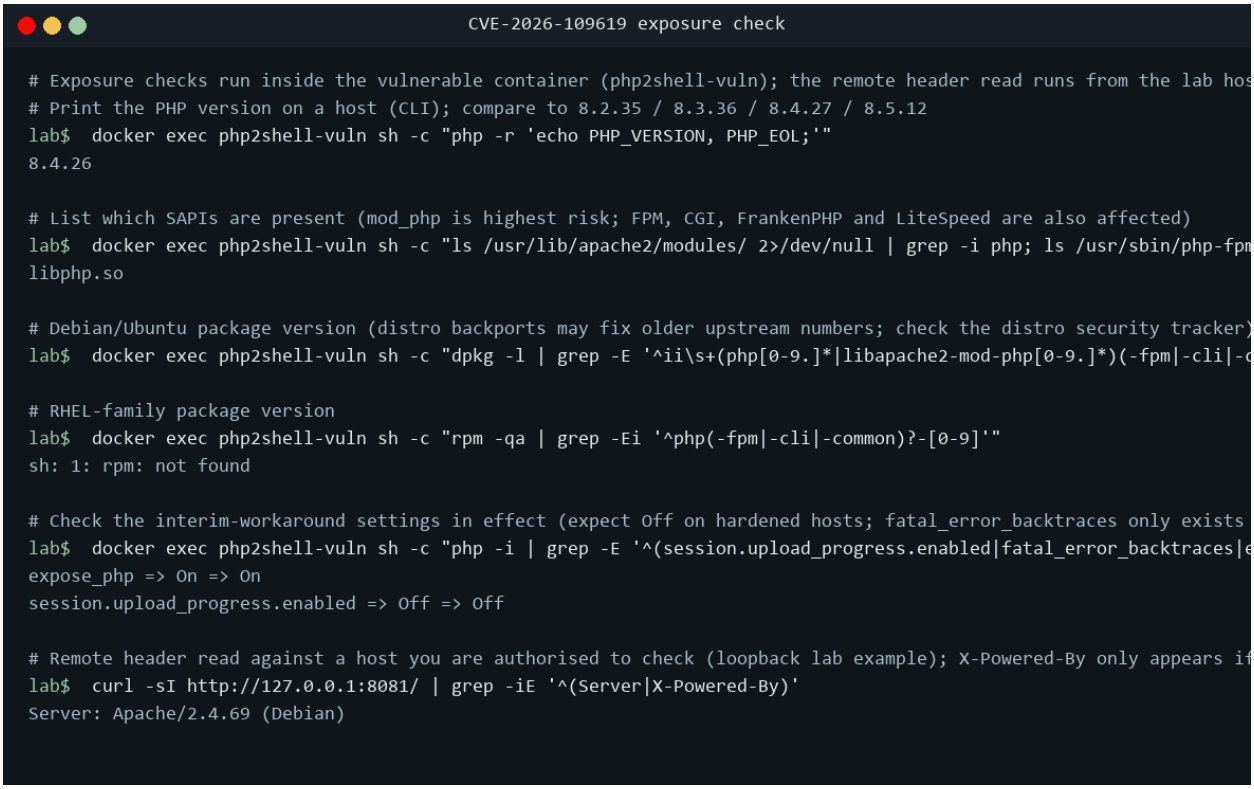
- `php -v` printed `PHP 8.4.26 (cli) (built: Oct 21 2026` in the vulnerable container and `PHP 8.4.27 (cli) (built: Oct 28 2026` in the fixed one; the Zend Engine line showed v4.4.26 against v4.4.27.
- HEAD requests to both containers returned `HTTP/1.1 200 OK` and `Server: Apache/2.4.69 (Debian)`. The vulnerable one on 127.0.0.1:8081 added `X-Powered-By: PHP/8.4.26`; the fixed one on 127.0.0.1:8082 added `X-Powered-By: PHP/8.4.27`.
- The marker page put both builds on one screen: `VULN: LAB-MARKER php=8.4.26` and `FIXED: LAB-MARKER php=8.4.27`.
- The Apache error log also exposes the version at start-up and again after the graceful restart: `AH00163: Apache/2.4.69 (Debian) PHP/8.4.26 configured`.

The `X-Powered-By` header is the one remote observable here, and it is only as good as the server's `expose_php` setting and the URL you ask for (see the exposure check below). `php -v` and the error log need a shell on the host or in the container, and the marker page is a lab-planted file with no counterpart on a real host (**Inferred**).

Exposure check

The exposure checks show what an inventory finds, and what it misses (**Demonstrated** [46] [47], `03-exposure.txt`):

- **A container sweep reported each running container's PHP.** A loop over `docker ps -q` with `docker exec $c php -r 'echo PHP_VERSION;'` returned one version per running container, `8.4.27` and `8.4.26` [46].
- **The package manager does not see the official image.** Inside `php:8.4.26-apache`, `dpkg -l` returned no PHP rows and `rpm` was absent (`sh: 1: rpm: not found`). Only `php -v`, `libphp.so` and the header revealed the vulnerable build [47].
- **A header check against `/` can come back empty.** On the CVE-2026-109619 target, `curl -sI` against `/` returned only `Server: Apache/2.4.69 (Debian)` and no `X-Powered-By`, even though `expose_php` was On. No PHP script was served at `/`; the CVE-2026-109618 target, which had `index.php`, did send the header at `/` [47] [46]. A remote check has to request a URL that PHP actually handles.
- **The lab host itself carried vulnerable packages.** The sweep on the lab host found `php8.4` packages at `8.4.24-1`, including `libapache2-mod-php8.4`, below 8.4.27. No Apache PHP module was loaded and no PHP-FPM, FrankenPHP or LSAPI process was running [46]. The plan's changelog check for a backport does not appear in the capture, so whether that package carries one was not checked. It is likely unpatched: Debian's tracker lists the trixie build of the same upstream release, `8.4.24-1~deb13u1`, as vulnerable, though the host's `8.4.24-1` is not that exact build (**Inferred** [31]); the lab host distribution's own tracker was not checked.



Capture `03-exposure.txt` of the CVE-2026-109619 run [47], 4 November 2026 UTC: version, module, package, settings and header checks, all read-only. Long lines are cut at the image's right edge.

The CVE-2026-109618 setup and cleanup screenshots match the CVE-2026-109619 ones shown here; its exposure capture lists the lab host's own packages and is quoted in the text above rather than shown.

The interim workarounds

Before mitigation the vulnerable container ran the image defaults: `memory_limit` 128M, `post_max_size` 8M and `session.upload_progress.enabled` On (**Demonstrated** [46], `02-test.txt`).

CVE-2026-109618 run. The plan wrote the workaround as Apache admin values in `/etc/apache2/conf-available/php2shell-mitigation.conf` (`php_admin_value memory_limit -1`, `php_admin_value post_max_size 1M`, `php_admin_flag session.upload_progress.enabled off`), enabled it with `a2enconf` and applied it with `apachectl -k graceful`. A read through the web server API then returned `-1 1M '0'` (**Demonstrated** [46], `04-mitigation.txt`). The mitigated host still sent `X-Powered-By: PHP/8.4.26`: the workaround changes settings, not the version, so a version scanner correctly keeps flagging the host, and only the upgrade clears the flag (**Demonstrated** [46]).

CVE-2026-109618 mitigation verification

```
lab$  docker exec php2shell-vuln sh -c "grep -rEi 'php_admin_(value|flag) +(memory_limit|session.upload_progress.ena
/etc/apache2/conf-available/php2shell-mitigation.conf:php_admin_value memory_limit -1
/etc/apache2/conf-available/php2shell-mitigation.conf:php_admin_flag session.upload_progress.enabled off

lab$  docker exec php2shell-vuln sh -c "ls -l /etc/apache2/conf-enabled/ | grep php2shell"
lrwxrwxrwx 1 root root 43 Nov  4 04:55 php2shell-mitigation.conf -> ../conf-available/php2shell-mitigation.conf

lab$  curl -s http://127.0.0.1:8081/ini.php
-1 1M '0'

lab$  curl -sI http://127.0.0.1:8081/index.php | grep -iE '^(HTTP|Server|X-Powered-By)'
HTTP/1.1 200 OK
Server: Apache/2.4.69 (Debian)
X-Powered-By: PHP/8.4.26

lab$  curl -sI http://127.0.0.1:8082/index.php | grep -iE '^(HTTP|Server|X-Powered-By)'
HTTP/1.1 200 OK
Server: Apache/2.4.69 (Debian)
X-Powered-By: PHP/8.4.27
```

Capture `04-mitigation.txt` of the CVE-2026-109618 run [46], 4 November 2026 UTC: the Apache admin values in place and the vulnerable version still advertised.

CVE-2026-109619 run. The plan wrote `session.upload_progress.enabled=0` to `/usr/local/etc/php/conf.d/zz-php2shell-mitigation.ini` and restarted Apache gracefully. The web page then printed `upload_progress=OFF`, and CLI `php -i` showed Off as well (**Demonstrated** [47], `04-mitigation.txt`). The fixed 8.4.27 container, left at its defaults, still had `session.upload_progress.enabled => On => On`: the upgrade does not change this default (**Demonstrated** [47]), while php.net and NCSC-NL advise keeping it off until the next bugfix release (**Confirmed** [11] [20]).

CVE-2026-109619 mitigation verification

```
# Read-only verification of the interim workaround (session.upload_progress.enabled=0) applied in T8
lab$  docker exec php2shell-vuln cat /usr/local/etc/php/conf.d/zz-php2shell-mitigation.ini
session.upload_progress.enabled=0

lab$  docker exec php2shell-vuln sh -c "php -i | grep -E '^session.upload_progress.enabled'"
session.upload_progress.enabled => Off => Off

lab$  curl -s http://127.0.0.1:8081/ini.php; echo
upload_progress=OFF

# Fixed container left at default for comparison (no workaround applied there; the vendor fix is the upgrade itself)
lab$  docker exec php2shell-fixed sh -c "php -i | grep -E '^session.upload_progress.enabled'"
session.upload_progress.enabled => On => On

lab$  curl -sI http://127.0.0.1:8082/version.php | grep -iE '^(HTTP|Server|X-Powered-By)'
HTTP/1.1 200 OK
Server: Apache/2.4.69 (Debian)
X-Powered-By: PHP/8.4.27
```

Capture `04-mitigation.txt` of the CVE-2026-109619 run [47], 4 November 2026 UTC: upload progress off in the vulnerable container and still on by default in the fixed one.

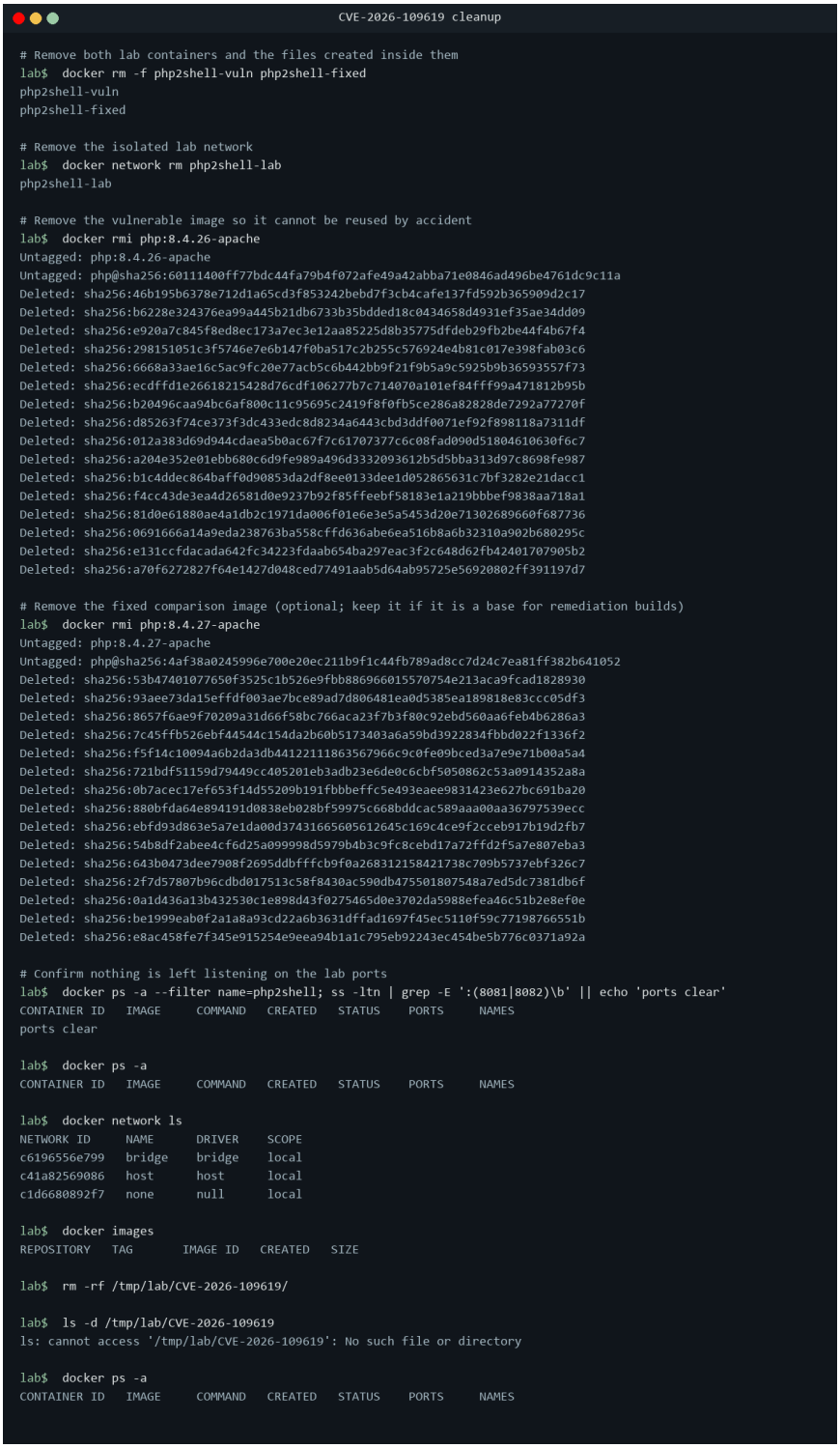
One caution comes from putting the two runs side by side. The CVE-2026-109618 run used Apache `php_admin_*` lines, which is the directory-configuration channel the CVE-2026-109619 advisory names as an early consumer when a value emits a diagnostic [2]. The lab's values were valid and the error log shows no PHP warning, so this is not a flaw in the run. The lesson is to keep such lines valid, or to put the settings in `php.ini` or the FPM pool instead (**Inferred** [2] [46]).

Detection observables in the lab

The lab logs recorded version-check traffic only: HEAD and GET lines from `curl/8.22.0` returning 200 for `/index.php`, `/ini.php` and `/version.php`, and `AH00171: Graceful restart requested, doing restart` at the moment each workaround was applied, followed by a fresh `AH00163 ... PHP/8.4.26 configured` line (**Demonstrated** [46] [47], `02-test.txt`). That is the expected result for a recognition-only lab: nothing in it would trip an exploitation rule, and the graceful-restart line is a useful change marker when you audit when a workaround went in (**Inferred**).

Cleanup

Both runs removed their containers and images, and the CVE-2026-109619 run removed its lab network. `docker ps -a` and `docker images` came back empty, the port check printed `ports clear` for 8081 and 8082, and the lab directories were deleted (**Demonstrated** [46] [47], `05-cleanup.txt`).



Capture `05-cleanup.txt` of the CVE-2026-109619 run [47], 4 November 2026 UTC.

What did not go to plan

Each departure from the written plans is recorded here, because each one carries a lesson for anyone repeating the run.

1. **The two plans collided.** Both used the same container names and ports, so the first `docker run` failed with a name conflict. Each run removed only its own containers and was then re-run cleanly (**Demonstrated** [46] [47], `00-attempt1-setup.txt`). Give concurrent lab plans distinct names and ports.
2. **A registry pull failed once.** A re-pull of `php:8.4.26-apache` in the CVE-2026-109619 run hit a transient `connection reset by peer`; the retry succeeded and returned the same digest (**Demonstrated** [47], `01-setup.txt`). Record digests, not only tags, so a retry can be proven identical.
3. **The CLI is not the web server API.** The CVE-2026-109618 plan expected its SAPI check to print `apache2handler`; it printed `cli`, because `docker exec ... php -r` runs the command-line binary. mod_php was proven through `apachectl -M` instead (**Demonstrated** [46], `02-test.txt`). On a real host, read settings through a page the web server serves, or from the loaded module, not from the CLI alone.
4. **The isolation was weaker than planned.** The CVE-2026-109619 run attached both containers to the default bridge as well as the `--internal` `php2shell-lab` network so the loopback publish would work, which weakened the planned "no outbound route" isolation; the access-log client was `172.17.0.1` (**Inferred** from the commands in [47], `01-setup.txt`). For an exploit-class run, use a network that blocks egress and publishes ports without a second bridge.
5. **The header and the host packages.** The header check at `/` and the lab host's own `8.4.24-1` packages are covered in the exposure check above (**Demonstrated** [46] [47]).

Part 4: finding and fixing it on real hosts

The October wave study carries the per-platform update commands [44]. This Part adds what the lab showed about finding PHP, and the fixed builds each distribution lists.

Step 1: find every PHP that serves requests

Run these where you are authorized to check. Each was run in the lab, on the lab host or inside a container (**Demonstrated** [46] [47]); in the last line, put a PHP-served URL of your own host in place of the lab's loopback address:

```
php -v
php -r 'echo PHP_VERSION, " ", php_sapi_name(), PHP_EOL;'
apachectl -M 2>/dev/null | grep -i php || httpd -M 2>/dev/null | grep -i php
ls /usr/lib/apache2/modules/ | grep -i php
ps -eo user,comm | grep -E 'php-fpm|frankenphp|lsphp'
dpkg -l | grep -E '^ii +(php|libapache2-mod-php)'
for c in $(docker ps -q); do echo "$c $(docker exec $c php -r 'echo PHP_VERSION;' 2>/dev/null)";
done
curl -sI http://127.0.0.1:8081/index.php | grep -iE '^(HTTP|Server|X-Powered-By)'
```

Read the results with the lab's lessons in mind:

- `php -v` and `php -r` report the command-line binary: the SAPI check printed `cli` (**Demonstrated** [46]). The web server may load a different build, so pair them with the module listing or the process list (**Inferred**).
- A package query finds distribution packages only. PHP in the official `php` Docker image is not a Debian package: `dpkg -l` inside one returned no PHP rows (**Demonstrated** [47]). Sweep containers with `docker exec`, and scan images, not only hosts.

- A header read needs a URL that PHP serves: `/` returned no `X-Powered-By` while `expose_php` was On (**Demonstrated** [47]). It also needs `expose_php` On, so an empty result proves nothing (**Inferred**).
- Products that embed PHP may not show up in any of these (**Inferred**). NCSC-NL tracks 930 such products, 215 of them vulnerable and 167 fixed as of 4 November 2026, and NVD's CPEs name Siemens and Palo Alto products (**Confirmed** [20] [5]).

Step 2: compare with the fixed build for your platform

Backported distribution packages keep their old upstream version strings, so compare the package release with the vendor's fixed build, not the upstream number with 8.4.27 (**Confirmed** for the versions; **Inferred** for the rule).

Platform	Fixed build	Grade	Source
Upstream, source and Windows builds	8.2.35, 8.3.36, 8.4.27, 8.5.12 (8.6.0 RC4 for testers); the Windows `-1` rebuilds report the same runtime version	Confirmed	[3] [10] [12]
Official Docker images	Rebuild from `php:8.4.27-apache` (or the matching tag of your branch) pinned by digest; the lab's fixed digest `sha256:4af38a024599...641052` matched the live tag on 4 November	Demonstrated and Confirmed	[30] [46]
Debian 13 (trixie)	`php8.4` 8.4.27-1~deb13u1 (DSA-6574-1); 8.4.24-1~deb13u1 is vulnerable	Confirmed	[31]
Debian 12 (bookworm)	`php8.2` 8.2.35-1~deb12u1 (DLA-4848-1)	Confirmed	[31]
Ubuntu 26.10	`php8.5` 8.5.9-0ubuntu2.1	Confirmed	[32]
Ubuntu 26.04 (resolute)	`php8.5` 8.5.4-0ubuntu1.5	Confirmed	[32]
Ubuntu 24.04 (noble)	`php8.3` 8.3.6-0ubuntu0.24.04.13	Confirmed	[32]
Ubuntu 22.04 (jammy)	`php8.1` 8.1.2-1ubuntu2.28	Confirmed	[32]
Ubuntu 14.04 and 16.04	Fixed through Ubuntu Pro Legacy	Confirmed	[32]
RHEL 10	`php` 8.3.36-1.el10_2 (RHSA-2026:87070); `php8.4` 8.4.27-1.el10_2 (RHSA-2026:87071)	Confirmed	[33]
RHEL 9	`php` 8.0.30-8.el9_8.1 (RHSA-2026:87151) plus the `php:8.2` and `php:8.3` module streams; the `php:8.4` stream is listed Affected	Confirmed	[33]
RHEL 7	Not affected by CVE-2026-109618; CVE-2026-109619 fixed in ELS `php` 5.4.16-48.el7_9.1 (RHSA-2026:88693)	Confirmed	[34]
cPanel EasyApache 4	NCSC-NL provides mitigation material	Confirmed	[20]

Red Hat rates both CVEs Important and notes that standard RHEL deployments use PHP-FPM rather than mod_php, which makes remote code execution harder in practice (**Confirmed** [33] [34]). That lowers the urgency on default RHEL web servers, not the need to update: the 109619 RCE was validated on FPM [2].

Step 3: update, restart and keep upload progress off

Install the fixed build with your platform's package manager or image rebuild (the wave study has the commands [44]), then restart every PHP worker, because a long-running worker keeps the old interpreter in memory (**Inferred**). Afterwards, check again: `php -v` and the module or process for the new version, the header from a PHP-served URL, and the upload-progress value read through a PHP-served page or the FPM pool configuration (with `php -i` only as a CLI cross-check) for Off. Set `session.upload_progress.enabled=0` in `php.ini`, the FPM pool or the server configuration, not in `ini_set()` or `.user.ini`, and keep it there until the next regular bugfix release [9] [11] [20].

Step 4: if you cannot update yet

The vendor's interim guidance is per CVE, and neither set is a complete fix (**Confirmed**):

- **CVE-2026-109618.** Enforce `memory_limit=-1` administratively and restart every worker; keep `post_max_size` low (8M without uploads, 32M with) and `upload_max_filesize` below it; bound nesting, parts and headers at the frontend; use a cgroup limit as an availability boundary; never use `RLIMIT_AS` or `RLIMIT_DATA` [1] [9].
- **CVE-2026-109619.** Set `session.upload_progress.enabled=0` administratively and restart every worker; on 8.5, set `fatal_error_backtraces=0`; optionally set `pm.max_requests=1`. Changing `error_reporting` or `display_errors`, or using `@`, does not protect. The vendor says no complete generic configuration mitigation exists [2].

The lab showed that both workarounds take effect through the web server API after a graceful restart (**Demonstrated** [46] [47]). In the CVE-2026-109618 run, whose workaround also turned upload progress off, the header still reported 8.4.26 afterwards (**Demonstrated** [46]). CISA adds that a web application firewall may provide some protection in the interim, and that organizations that cannot update immediately, or that still run PHP 8.1 or earlier, should take PHP-based systems offline until they are patched (**Confirmed** [17]). A `memory_limit=-1` setting removes PHP's own memory ceiling, so the cgroup limit is what keeps one request from exhausting the host (**Inferred** from [1]).

Step 5: detect and investigate

- **Watch the worker processes.** CISA tells defenders to watch for PHP worker processes launching shells or downloading tools (**Confirmed** [17]).
- **Network signatures exist, with caveats.** CISA's SIGMA rule, `CISA_SIGMA_261429_CVE_2026_109618-109619_PHP2Shell_TLP_CLEAR.yaml` in cisagov/SIGMA_Rules, is dated 21 October, modified 23 October, status test, level critical; it matches keywords over request, error and indicator strings, and CISA states its false-positive rate as medium-high (**Confirmed** [35]). Corelight's php2shell-detect ships a Zeek package and Suricata rules, with notices including `Multipart_Header_Flood`, `Large_Form_Key`, serialized-data patterns and `Potentially_Affected_PHP_Version`, raised from response headers (**Confirmed** [36]). NCSC-NL also lists a community local-inventory tool that it says it has not reviewed (**Confirmed** [20]).

- **Look back far enough.** KEV marks both entries `forensicTriage` Yes [18], and Amazon advises investigating from 17 October [22]. TU Delft counted 84,889 exploitation requests from 797 source IPs between 21 October and 4 November, 181 of those IPs sending code or a dropper and 2 sending 49 requests before the fixed releases (**Reported** [23]).
- **Use the server's own logs as change markers.** In the lab, `AH00163` lines carried the PHP version at start-up and after the graceful restart, and `AH00171` marked the graceful restart (**Demonstrated** [46] [47]). A worker restart that nobody scheduled, or a version line that changes back, is worth a look (**Inferred**).

Part 5: the vulnerability-management view

The two CVEs are a clean example of signals that all point the same way, and of the ways an inventory still gets it wrong.

- **Severity, exploitation and automation agree on urgency.** CNA 9.2 for both, NVD 9.8 and 8.1, SSVC "exploitation active" and "technical impact total", a KEV listing five days after release with a three-day due date (**Confirmed** [5] [6] [18]). SSVC also records "automatable no", and NVD gives 109619 `AC:H`, so exploitation likely takes per-target effort (**Inferred**). None of that changes the action.
- **The scanner is right about mitigated hosts.** The lab's mitigated container still advertised 8.4.26 (**Demonstrated** [46]). Record workarounds as risk acceptance with an end date, not as remediation, so the finding stays open until the upgrade.
- **The scanner is wrong about backported hosts.** Ubuntu noble's fixed package is 8.3.6-based and RHEL 9's default `php` package is 8.0.30-based (**Confirmed** [32] [33]). A banner or header check will flag them; the package release or changelog decides (**Inferred**).
- **The scanner cannot see what it cannot reach.** A header read against `/` missed the version, and the package manager missed the Docker image (**Demonstrated** [47]). Asset context has to come from container and image inventory as well as host agents.
- **Range data needs care.** The CNA's range stops at 8.2; NVD's goes back to 5.4.0 for 109619 (**Confirmed** [3] [6]). A rule built only from the CNA range would mark end-of-life PHP as unaffected, which is the opposite of CISA's advice that systems which cannot update immediately, or which run PHP 8.1 or earlier, be taken offline until patched [17] (**Inferred**).
- **Images move under their tags.** The fixed `php:8.4.27-apache` tag was last pushed on 31 October, ten days after the PHP release, and its amd64 image is dated 28 October (**Confirmed** [30]). Pin base images by digest and record the digest in the finding's evidence.

See the Vulnerability Prioritization Reference and the Vulnerability Management Reference for the method these signals feed.

Part 6: MITRE mappings

ATT&CK has no group, software or campaign object for these CVEs in the library's v20.0 data, so the bindings are technique-level. The ids below were checked against the live ATT&CK, CWE, CAPEC and D3FEND pages or the library's data files on 4 November 2026 [37] to [43] [45] [48] [49].

ATT&CK Enterprise techniques

ATT&CK id	Name	Tactic (v20)	Dated event or report	Basis	Source
T1190	Exploit Public-Facing Application	Initial Access	Unauthenticated RCE over HTTP validated by the vendor (2026-10-21); KEV listing (2026-10-26)	Confirmed: the record supports it directly	[1] [2] [16] [18] [39]
T1059.004	Command and Scripting Interpreter: Unix Shell	Execution	CISA alert, PHP workers launching shells (2026-10-21)	Inferred follow-on	[17] [40]
T1105	Ingress Tool Transfer	Command and Control	CISA alert, PHP workers downloading tools (2026-10-21); TU Delft count of IPs sending code or a dropper (to 2026-11-04, Reported)	Inferred follow-on	[17] [23] [45] [49]
T1505.003	Server Software Component: Web Shell	Persistence	RCE as the web worker account (2026-10-21)	Inferred follow-on; no source opened for this study records a web shell	[1] [2] [45] [48]
T1499.004	Endpoint Denial of Service: Application or System Exploitation	Impact	OOM producers (109618) and worker-state corruption (109619), vendor advisories (2026-10-21)	Inferred side effect	[1] [2] [41]

ATT&CK for ICS and Mobile do not apply. The library's CAPEC data maps CAPEC-130 to T1499.003 Application Exhaustion Flood; that id is context only and is not bound here, because no source describes the OOM requests as a flood [45].

Group, software and campaign objects

Object	Id	Relation to this study	Source
Groups	none	No ATT&CK group in the library's v20.0 data is tied to these CVEs; the actors behind the attempts in the telemetry are not named in the sources opened	[45]
Software	none	No ATT&CK software object covers php2shell	[45]
Campaigns	none	No ATT&CK campaign covers php2shell	[45]

CWE and CAPEC

Id	Name	Where it appears	Basis	Source
CWE-416	Use After Free	Advisories, KEV and NVD, both CVEs	Confirmed	[1] [2] [5] [6] [18]
CWE-755	Improper Handling of Exceptional Conditions	Advisories, KEV and NVD, both CVEs	Confirmed	[1] [2] [5] [6] [18]
CWE-824	Access of Uninitialized Pointer	Advisories, KEV and NVD, both CVEs	Confirmed	[1] [2] [5] [6] [18]
CWE-1325	Improperly Controlled Sequential Memory Allocation	Advisories, KEV and NVD, both CVEs	Confirmed	[1] [2] [5] [6] [18] [37]
CWE-1419	Incorrect Initialization of Resource	Advisories and KEV only, both CVEs	Confirmed	[1] [2] [18]
CAPEC-130	Excessive Allocation	Linked from the CWE-1325 page; the library's CAPEC data gives related CWE-1325. It fits the OOM-producing request class of CVE-2026-109618, not the use after free	Mapping Confirmed; fit Inferred	[37] [38] [45]

No CAPEC entry is linked from CWE-416, 755, 824 or 1419 [37] [38]. The library has rows for CWE-755 and CWE-1419 in the CWE Reference and no per-id CWE pages.

D3FEND countermeasures

Countermeasure	What it covers here	Basis	Source
Software Update (D3-SU)	The upgrade, the only complete fix	Inferred; the library's data does not map it to these techniques	[42] [8]
Application Configuration Hardening (D3-ACH)	The interim `php.ini`, pool and server settings	Inferred; not mapped in the library's data	[1] [2] [45]
Inbound Traffic Filtering (D3-ITF)	Frontend limits on parts, headers and nesting; CISA's interim WAF note	The library maps it to T1190	[45] [1] [17]
Process Spawn Analysis (D3-PSA)	PHP workers launching shells	The library maps it to T1505.003	[43] [45] [17]
File Integrity Monitoring (D3-FIM)	New scripts written into the document root after compromise	The library maps it to T1505.003	[45]

Navigator layer

The study's techniques are saved as an ATT&CK Navigator layer, navigator/library-layers/php-8-4-27-cves.json, listed on the Navigator layers page. It is pinned to ATT&CK v20, domain `enterprise-attack`. Score 100 marks the technique the record supports directly (T1190); score 50 marks the study's labelled inferences. Each entry's comment names the source numbers above.

MITRE ATT&CK, CWE, CAPEC and D3FEND content is used under the terms recorded in THIRD_PARTY_NOTICES.md; the notices close the Sources part.

Cross-cutting themes

- **Failure paths are attack surface.** Both flaws live in what PHP does when something goes wrong: an out-of-memory abort, a fatal error, a late diagnostic. The fix redesigned the failure path rather than patching a parser [1] [2] [14]. Error handling deserves the same review as input handling (**Inferred**). The same lesson runs through the Log4Shell study, where a logging feature was the entry point.
- **The runtime is under the application.** Wiz reports pre-authentication RCE against WordPress, Drupal and Joomla container images (**Reported** [25]), and on mod_php one flaw fires before any application code runs (**Confirmed** [1]). Application inventories that stop at the CMS version miss it (**Inferred**).
- **A workaround is a setting, not a version.** The lab showed both interim workarounds working [46] [47] and, in the CVE-2026-109618 run, the vulnerable version still advertised [46]. Programs need a state for "mitigated, not remediated" that a scanner cannot close on its own (**Inferred**).
- **Inventory is a set of methods, not a tool.** The package manager, the header and the CLI each missed something in a two-container lab [46] [47]. Combining host packages, loaded modules and container sweeps found every build in the lab (**Demonstrated** [46] [47]); image scans would add images that are not running (**Inferred**).
- **Disclosure is a process that can leak.** The fix stalled for months, which the PHP Foundation calls a structural failure; then, in the last day, the CVE ids leaked with the wrong score, a demo video was published by mistake and fixed images went out before the coordinated time [21]. Amazon reports an attempt against its honeypots on 17 October, three days before the leak (**Reported** [22]). Plan emergency patching on the assumption that the public clock starts before the advisory (**Inferred**).
- **What is overstated, and what is understated.** Overstated: the leaked 10.0 score, which no record carries [21] [5] [6], and the circulating proof-of-concept code, none of which the PHP Foundation had identified as functional at release time [21]. Understated: end-of-life PHP and products that embed PHP, which the CNA range does not list and which NCSC-NL counts in the hundreds [3] [20]; and the follow-on CVE-2026-115560, which is why upload progress should stay off after updating [11].

Sources

Sources 1 to 45, 48 and 49 were opened on 4 November 2026; sources 46 and 47 are the lab captures.

#	Source	URL	Opened	Grade
1	PHP security advisory GHSA-w8hw-w9fx-j79w, "RCE via OOM corruption" (CVE-2026-109618)	https://github.com/php/php-src/security/advisories/GHSA-w8hw-w9fx-j79w	2026-11-04	Confirmed
2	PHP security advisory GHSA-6wmf-4rv5-p3rp, "RCE via stale global state" (CVE-2026-109619)	https://github.com/php/php-src/security/advisories/GHSA-6wmf-4rv5-p3rp	2026-11-04	Confirmed
3	CVE record CVE-2026-109618 (PHP CNA)	https://cveawg.mitre.org/api/cve/CVE-2026-109618	2026-11-04	Confirmed
4	CVE record CVE-2026-109619 (PHP CNA)	https://cveawg.mitre.org/api/cve/CVE-2026-109619	2026-11-04	Confirmed
5	NVD record CVE-2026-109618 (API)	https://services.nvd.nist.gov/rest/json/cves/2.0?cveId=CVE-2026-109618	2026-11-04	Confirmed

#	Source	URL	Opened	Grade
6	NVD record CVE-2026-109619 (API), including the CISA ADP SSVC decision	https://services.nvd.nist.gov/rest/json/cves/2.0?cveId=CVE-2026-109619	2026-11-04	Confirmed
7	php.net news archive, advance notice of the security release (2026-10-20)	https://www.php.net/archive/2026.php#2026-10-20-1	2026-11-04	Confirmed
8	php.net news archive, "PHP 8.4.27 Released!" (2026-10-21)	https://www.php.net/archive/2026.php#2026-10-21-3	2026-11-04	Confirmed
9	php.net news archive, security advisory (2026-10-21)	https://www.php.net/archive/2026.php#2026-10-21-5	2026-11-04	Confirmed
10	php.net news archive, PHP 8.6.0 RC4 (2026-10-21)	https://www.php.net/archive/2026.php#2026-10-21-6	2026-11-04	Confirmed
11	php.net news archive, informational advisory for CVE-2026-115560 (2026-10-23)	https://www.php.net/archive/2026.php#2026-10-23-1	2026-11-04	Confirmed
12	php.net news archive, Windows rebuilds with Brotli 1.2.1 (2026-10-26)	https://www.php.net/archive/2026.php#2026-10-26-1	2026-11-04	Confirmed
13	PHP 8 ChangeLog, Version 8.4.27	https://www.php.net/ChangeLog-8.php#8.4.27	2026-11-04	Confirmed
14	php-src commit 3584a98bda0f, "Avoid state corruption via bailout"	https://github.com/php/php-src/commit/3584a98bda0f2c0d367255b046798b0d3a221a19	2026-11-04	Confirmed
15	oss-security, PHP security team post (2026-10-21)	https://www.openwall.com/lists/oss-security/2026/10/21/6	2026-11-04	Confirmed
16	oss-security, PHP security team post (2026-10-21)	https://www.openwall.com/lists/oss-security/2026/10/21/7	2026-11-04	Confirmed
17	CISA alert, "Critical Vulnerabilities in PHP Require Immediate Patching" (2026-10-21, revised 2026-10-23 and 2026-10-26)	https://www.cisa.gov/news-events/alerts/2026/10/21/critical-vulnerabilities-php-require-immediate-patching	2026-11-04	Confirmed
18	CISA Known Exploited Vulnerabilities catalog, JSON feed (catalog version 2026.11.03)	https://www.cisa.gov/sites/default/files/feeds/known_exploited_vulnerabilities.json	2026-11-04	Confirmed
19	CERT-EU security advisory 2026-018 (v1.0 to v1.2)	https://cert.europa.eu/publications/security-advisories/2026-018/	2026-11-04	Confirmed
20	NCSC-NL php2shell repository (product list, detection and mitigation directory)	https://github.com/NCSC-NL/php2shell	2026-11-04	Confirmed
21	The PHP Foundation, "Today's PHP security release and how we're strengthening PHP security" (2026-10-21)	https://thephp.foundation/blog/2026/10/21/todays-php-security-release-and-how-were-strengthening-php-security/	2026-11-04	Confirmed
22	Amazon Threat Intelligence, php2shell exploitation prior to public disclosure in MadPot	https://aws.amazon.com/blogs/security/amazon-threat-intelligence-identifies-php2shell-exploitation-prior-to-public-disclosure-in-madpot/	2026-11-04	Reported
23	TU Delft, Delft Threat Intelligence Lab, php2shell exploitation counts	https://php2shell.delftintellab.com/	2026-11-04	Reported

#	Source	URL	Opened	Grade
24	watchTowr, php2shell	https://watchtowr.com/intelligence/php-remote-code-execution-php2shell/	2026-11-04	Reported
25	Wiz Research, php2shell	https://www.wiz.io/blog/php2shell-php-rce	2026-11-04	Reported
26	Calif, php2shell write-up	https://calif.io/php2shell/	2026-11-04	Reported
27	php-src issue GH-24492 (community-filed, open)	https://github.com/php/php-src/issues/24492	2026-11-04	Reported
28	php-src issue GH-24493 (community-filed, open)	https://github.com/php/php-src/issues/24493	2026-11-04	Reported
29	Docker Hub, `php:8.4.26-apache` tag	https://hub.docker.com/v2/repositories/library/php/tags/8.4.26-apache	2026-11-04	Confirmed
30	Docker Hub, `php:8.4.27-apache` tag	https://hub.docker.com/v2/repositories/library/php/tags/8.4.27-apache	2026-11-04	Confirmed
31	Debian security tracker, CVE-2026-109618 and CVE-2026-109619	https://security-tracker.debian.org/tracker/CVE-2026-109618 and https://security-tracker.debian.org/tracker/CVE-2026-109619	2026-11-04	Confirmed
32	Ubuntu security, CVE-2026-109618	https://ubuntu.com/security/CVE-2026-109618	2026-11-04	Confirmed
33	Red Hat security data, CVE-2026-109618	https://access.redhat.com/hydra/rest/securitydata/cve/CVE-2026-109618.json	2026-11-04	Confirmed
34	Red Hat security data, CVE-2026-109619	https://access.redhat.com/hydra/rest/securitydata/cve/CVE-2026-109619.json	2026-11-04	Confirmed
35	cisagov/SIGMA_Rules, php2shell rule	https://github.com/cisagov/SIGMA_Rules	2026-11-04	Confirmed
36	Corelight php2shell-detect	https://github.com/corelight/php2shell-detect	2026-11-04	Confirmed
37	MITRE CWE, CWE-1325	https://cwe.mitre.org/data/definitions/1325.html	2026-11-04	Confirmed
38	MITRE CAPEC, CAPEC-130	https://capec.mitre.org/data/definitions/130.html	2026-11-04	Confirmed
39	MITRE ATT&CK, T1190	https://attack.mitre.org/techniques/T1190/	2026-11-04	Confirmed
40	MITRE ATT&CK, T1059.004	https://attack.mitre.org/techniques/T1059/004/	2026-11-04	Confirmed
41	MITRE ATT&CK, T1499.004	https://attack.mitre.org/techniques/T1499/004/	2026-11-04	Confirmed
42	MITRE D3FEND API, Software Update	https://d3fend.mitre.org/api/technique/d3f:SoftwareUpdate.json	2026-11-04	Confirmed
43	MITRE D3FEND API, Process Spawn Analysis	https://d3fend.mitre.org/api/technique/d3f:ProcessSpawnAnalysis.json	2026-11-04	Confirmed

#	Source	URL	Opened	Grade
44	TeamStarWolf, PHP2Shell and the October 2026 Attack Wave	https://github.com/TeamStarWolf/TeamStarWolf/blob/main/case-studies/PHP2SHELL_OCTOBER_2026.md	2026-11-04	Confirmed
45	TeamStarWolf library data: `data/attack/technique_profiles.jsonl`, `data/attack/technique_to_d3fend.jsonl`, `data/weaknesses/capec.jsonl`, the campaign, group and software files under `data/attack/`, and the `mitre/` tree	https://github.com/TeamStarWolf/TeamStarWolf/tree/main/data	2026-11-04	Confirmed
46	TeamStarWolf lab run, CVE-2026-109618 plan: command output from `php:8.4.26-apache` and `php:8.4.27-apache` containers on a lab host (`01-setup.txt` to `05-cleanup.txt`), rendered as the screenshot in Part 3	none (lab capture)	2026-11-04 (UTC)	Demonstrated
47	TeamStarWolf lab run, CVE-2026-109619 plan: the same images (`00-attempt1-setup.txt` to `05-cleanup.txt`), rendered as the other screenshots in Part 3	none (lab capture)	2026-11-04 (UTC)	Demonstrated
48	MITRE ATT&CK, T1505.003	https://attack.mitre.org/techniques/T1505/003/	2026-11-04	Confirmed
49	MITRE ATT&CK, T1105	https://attack.mitre.org/techniques/T1105/	2026-11-04	Confirmed

© 2026 The MITRE Corporation. This work is reproduced and distributed with the permission of The MITRE Corporation. License: ATT&CK Terms of Use. ATT&CK® and MITRE ATT&CK® are registered trademarks of The MITRE Corporation. This project is not affiliated with or endorsed by MITRE.

© The MITRE Corporation. Includes material from the Common Weakness Enumeration (CWE™), used under the CWE Terms of Use. https://cwe.mitre.org/

© The MITRE Corporation. Includes material from the Common Attack Pattern Enumeration and Classification (CAPEC™), used under the CAPEC Terms of Use. https://capec.mitre.org/

© The MITRE Corporation. Includes material from MITRE D3FEND™, used under the D3FEND Terms of Use. https://d3fend.mitre.org/

The terms for every MITRE dataset used here are recorded in THIRD_PARTY_NOTICES.md.

Related references

- PHP2Shell and the October 2026 Attack Wave, the home for the October wave and the per-platform update commands
- PHP
- CVE Reference, CWE Reference, CAPEC Reference, D3FEND Reference
- Threat Actors, Malware Families, Notable Incidents
- Vulnerability Prioritization Reference, Vulnerability Management Reference, Triage a CVE
- Web Application Security Reference

- Other lab-backed studies: Ghostcat, CVE-2020-1938, ProFTPD mod_copy, CVE-2015-3306, Node.js, CVE-2026-111497, Magento and Adobe Commerce, CVE-2026-48356
- Other vulnerability studies: CVE-2026-98432 Linux TCP use-after-free, Log4Shell (CVE-2021-44228): The Case Study
- Techniques: T1190 Exploit Public-Facing Application, T1059.004 Unix Shell, T1105 Ingress Tool Transfer, T1505.003 Web Shell, T1499.004 Application or System Exploitation
- CAPEC: CAPEC-130 Excessive Allocation
- D3FEND: Software Update, Application Configuration Hardening, Inbound Traffic Filtering, Process Spawn Analysis, File Integrity Monitoring
- Navigator layers, including this study's layer, php-8-4-27-cves.json
- All case studies